

Plan complet et détaillé de réalisation — Connected Neighbours

0. Objectif du document

Ce document définit un plan complet pour réaliser le projet **Connected Neighbours** de manière cohérente, ambitieuse et défendable en soutenance.

L'objectif n'est pas de produire une simple suite de CRUD, mais une véritable plateforme de quartier avec :

- des workflows métier complets ;
 - des règles de gestion claires ;
 - des états maîtrisés ;
 - une traçabilité des actions ;
 - des contrats numériques ;
 - un système de points robuste ;
 - une messagerie liée aux interactions ;
 - une application JavaFX offline-first ;
 - une synchronisation entre client lourd et API ;
 - une architecture documentée et déployable.
-

1. Vision produit

1.1 Concept général

Connected Neighbours est une plateforme collaborative sécurisée de quartier permettant aux habitants de s'entraider, d'échanger des services, de signer des documents, de participer à la vie locale, de voter, de communiquer et de signaler des incidents.

La plateforme doit créer un cadre de confiance entre voisins.

Elle doit donc gérer :

- l'identité des utilisateurs ;
- les rôles et permissions ;
- les interactions entre habitants ;
- les engagements contractuels ;
- les preuves et signatures ;
- les paiements internes en points ;
- les litiges ;
- les signalements ;
- les événements ;
- les votes ;
- les documents ;

- la messagerie ;
 - les données personnelles ;
 - les statistiques ;
 - la synchronisation offline.
-

1.2 Problème à résoudre

Dans un quartier, les habitants peuvent vouloir :

- demander de l'aide ;
- proposer leurs compétences ;
- organiser des événements ;
- voter sur des décisions locales ;
- discuter entre voisins ;
- signaler des problèmes ;
- conserver une trace des engagements ;
- faire confiance à des personnes qu'ils connaissent peu.

Une simple application d'annonces ne suffit pas, car elle ne gère pas :

- les engagements ;
- les paiements en points ;
- les conflits ;
- les preuves ;
- les signatures ;
- la réputation ;
- la modération ;
- l'administration offline.

Connected Neighbours doit donc être pensé comme un **système de confiance communautaire**.

1.3 Objectifs fonctionnels

La plateforme doit permettre :

1. La gestion des quartiers géographiques.
2. L'inscription et la connexion des habitants.
3. La publication de services entre voisins.
4. La gestion des candidatures aux services.
5. La négociation des conditions d'un service.
6. La génération de contrats.
7. La signature numérique de documents.
8. Le paiement interne par points.
9. La gestion de litiges.
10. La création d'événements.
11. La recommandation via Neo4j.
12. La création de votes.
13. La messagerie multimédia.

14. La gestion des incidents et alertes.
 15. L'administration web.
 16. L'administration JavaFX offline-first.
 17. La synchronisation JavaFX/API.
 18. La conformité RGPD.
 19. La conteneurisation.
 20. La documentation et les tests.
-

1.4 Objectifs techniques

Le projet doit fournir :

- une API REST documentée avec Swagger ;
 - une application web React pour les habitants ;
 - une application React d'administration ;
 - une application JavaFX administrateur ;
 - une base MongoDB ;
 - une base Neo4j ;
 - un stockage MinIO pour les fichiers ;
 - une base locale H2 ou SQLite côté JavaFX ;
 - un système de synchronisation ;
 - un micro-langage MongoDB ;
 - un Docker Compose complet ;
 - des tests unitaires, intégration et E2E ;
 - un dossier technique ;
 - un dossier utilisateur ;
 - une synthèse critique.
-

2. Principes de conception

2.1 Ne pas faire de CRUD naïf

Chaque module doit être conçu comme un workflow.

Exemple incorrect :

```
Un voisin crée un service.  
Un voisin accepte.  
Le service est terminé.
```

Exemple attendu :

```
Un voisin publie une demande.  
Plusieurs voisins candidatent.  
Le demandeur compare les candidatures.
```

Les parties négocient les conditions.
Une proposition est acceptée.
Un contrat est généré.
Les points sont réservés.
Les deux parties signent.
Le service est planifié.
Le prestataire marque le service comme réalisé.
Le demandeur valide.
Les points sont transférés.
Les deux parties peuvent laisser un avis.
En cas de problème, un litige est ouvert.

2.2 Toute action importante doit être traçable

Les actions suivantes doivent être historisées :

- création d'un service ;
- modification d'un service ;
- candidature ;
- contre-proposition ;
- acceptation ;
- génération de contrat ;
- signature ;
- réservation de points ;
- transfert de points ;
- annulation ;
- litige ;
- décision de modération ;
- export RGPD ;
- synchronisation JavaFX ;
- résolution d'incident.

Chaque trace doit idéalement contenir :

```
id
actorId
action
targetType
targetId
previousValue
newValue
metadata
createdAt
ipAddress si disponible
userAgent si disponible
```

2.3 Priorité à la cohérence métier

Il vaut mieux avoir un parcours complet et solide sur les services qu'avoir dix modules superficiels.

Le cœur du projet doit être :

Services → Candidatures → Contrats → Points → Documents → Signatures →
Messagerie → Avis → Litiges

Les autres modules doivent s'intégrer autour de ce cœur.

2.4 Découpage par priorité

Chaque fonctionnalité doit être classée :

Priorité	Signification
P0	Indispensable pour soutenance
P1	Important pour montrer la profondeur
P2	Bonus / amélioration
P3	Hors périmètre réaliste

3. Acteurs et rôles

3.1 Habitant

Un habitant est l'utilisateur principal de la plateforme.

Il peut :

- consulter son quartier ;
 - créer une demande de service ;
 - publier une offre de service ;
 - candidater à un service ;
 - discuter avec d'autres habitants ;
 - signer un contrat ;
 - consulter ses points ;
 - participer à un événement ;
 - voter ;
 - signaler un contenu ;
 - signaler un incident ;
 - exporter ses données personnelles.
-

3.2 Demandeur de service

Un habitant devient demandeur lorsqu'il publie une demande.

Il peut :

- recevoir des candidatures ;
 - comparer les candidats ;
 - demander des précisions ;
 - accepter une candidature ;
 - signer le contrat ;
 - valider le service réalisé ;
 - ouvrir un litige ;
 - laisser un avis.
-

3.3 Prestataire voisin

Un habitant devient prestataire lorsqu'il répond à un service.

Il peut :

- candidater ;
 - proposer une date ;
 - proposer un prix ;
 - accepter un contrat ;
 - déclarer le service réalisé ;
 - ajouter une preuve ;
 - recevoir les points ;
 - ouvrir un litige ;
 - laisser un avis.
-

3.4 Modérateur

Le modérateur garantit le bon fonctionnement de la communauté.

Il peut :

- consulter les signalements ;
 - masquer un contenu ;
 - avertir un utilisateur ;
 - suspendre temporairement un utilisateur ;
 - intervenir dans un litige ;
 - consulter les preuves ;
 - prendre une décision de modération.
-

3.5 Administrateur web

L'administrateur web gère la plateforme.

Il peut :

- créer et modifier les quartiers ;
 - gérer les utilisateurs ;
 - gérer les rôles ;
 - consulter les statistiques globales ;
 - superviser les services ;
 - consulter les contrats ;
 - gérer les votes ;
 - gérer les événements ;
 - consulter les incidents ;
 - superviser la synchronisation JavaFX ;
 - consulter les exports RGPD.
-

3.6 Administrateur JavaFX

L'administrateur JavaFX travaille depuis une application desktop.

Il peut :

- consulter les incidents ;
 - créer des incidents hors ligne ;
 - gérer les alertes ;
 - consulter les statistiques locales ;
 - personnaliser l'interface ;
 - utiliser des plugins ;
 - synchroniser avec l'API centrale.
-

4. Architecture cible

4.1 Vue globale

```
[React Web Utilisateur]
  |
  | REST + WebSocket
  v
[API NestJS / Fastify]
  |
  |---- MongoDB
  |---- Neo4j
  |---- MinIO
  |---- Keycloak
```

```
      |  
      | REST Sync  
      v  
[JavaFX Admin Desktop]  
      |  
      v  
[Base locale H2/SQLite]
```

4.2 Applications du monorepo

apps/api	API centrale
apps/web	Front web habitant
apps/admin-web	Back-office React administrateur
apps/admin-desktop	Application JavaFX administrateur
docs	Documentation projet
diagrams	Schémas architecture
docker-compose.yml	Environnement complet

4.3 Bases de données

MongoDB

MongoDB stocke les données applicatives principales :

- utilisateurs ;
- quartiers ;
- services ;
- candidatures ;
- propositions ;
- contrats ;
- documents ;
- signatures ;
- points ;
- transactions ;
- événements ;
- votes ;
- messages ;
- notifications ;
- litiges ;
- signalements ;
- incidents ;
- alertes ;
- audit logs.

Neo4j

Neo4j stocke les relations sociales et les interactions :

```
(User)-[:LIVES_IN]->(Neighborhood)
(User)-[:CREATED]->(Service)
(User)-[:APPLIED_TO]->(Service)
(User)-[:HELPED]->(User)
(User)-[:REVIEWED]->(User)
(User)-[:PARTICIPATED_IN]->(Event)
(User)-[:INTERESTED_IN]->(Event)
(User)-[:VOTED_ON]->(Vote)
(Service)-[:IN_CATEGORY]->(Category)
(Event)-[:IN_CATEGORY]->(Category)
```

Neo4j sert aux recommandations :

- voisins fiables ;
- événements pertinents ;
- services compatibles ;
- catégories populaires ;
- interactions positives.

MinIO

MinIO stocke les fichiers :

- PDF originaux ;
- PDF signés ;
- images envoyées en messagerie ;
- vocaux ;
- pièces jointes ;
- preuves de litige ;
- documents d'incident.

MongoDB ne stocke que les métadonnées :

```
objectKey
bucket
filename
mimeType
size
ownerId
contextType
contextId
createdAt
```

Base locale JavaFX

La base locale stocke :

- incidents ;
 - alertes ;
 - reporters ;
 - statistiques locales ;
 - outbox de synchronisation ;
 - conflits de synchronisation ;
 - état de dernière synchronisation ;
 - préférences locales ;
 - thèmes ;
 - plugins.
-

5. Module Authentification et sécurité

5.1 Objectif

Sécuriser l'accès à la plateforme et protéger les actions sensibles.

5.2 Fonctionnalités P0

- Connexion utilisateur.
 - JWT local en développement.
 - Protection des routes.
 - Rôles.
 - Récupération du profil connecté.
 - Guard admin/modérateur.
-

5.3 Fonctionnalités P1

- Refresh token.
 - Expiration des sessions.
 - Déconnexion.
 - Changement de mot de passe.
 - Changement d'email avec vérification.
 - Journalisation des connexions.
-

5.4 Fonctionnalités P2

- Keycloak OIDC.
- SSO web + JavaFX.
- MFA TOTP.

- MFA obligatoire pour :
- signature ;
- suppression compte ;
- changement mot de passe ;
- changement email ;
- action admin critique.

5.5 Endpoints recommandés

```
POST /api/auth/login
POST /api/auth/logout
POST /api/auth/refresh
GET /api/auth/me
POST /api/auth/change-password
POST /api/auth/request-email-change
POST /api/auth/confirm-email-change
POST /api/auth/mfa/setup
POST /api/auth/mfa/verify
POST /api/auth/mfa/disable
```

6. Module Utilisateurs

6.1 Objectif

Gérer les comptes, profils, rôles et informations personnelles.

6.2 Données utilisateur

```
id
email
passwordHash
displayName
firstName
lastName
phone
avatarObjectKey
role
neighborhoodId
pointsBalance
reservedPoints
frozenPoints
reputationScore
isActive
```

```
createdAt
updatedAt
lastLoginAt
```

6.3 Fonctionnalités habitant

- Consulter son profil.
- Modifier son profil.
- Ajouter une photo.
- Voir son score de réputation.
- Voir ses statistiques personnelles.
- Exporter ses données.
- Demander suppression/anonymisation.

6.4 Fonctionnalités admin

- Lister les utilisateurs.
- Filtrer par quartier.
- Modifier le rôle.
- Désactiver un compte.
- Voir l'historique d'activité.
- Voir les litiges liés.
- Voir les signalements liés.

6.5 Endpoints recommandés

```
GET    /api/users/me
PATCH /api/users/me
GET    /api/users/me/stats
GET    /api/users/me/reputation

GET    /api/admin/users
GET    /api/admin/users/:id
PATCH /api/admin/users/:id
PATCH /api/admin/users/:id/role
PATCH /api/admin/users/:id/deactivate
PATCH /api/admin/users/:id/reactivate
```

7. Module Quartiers

7.1 Objectif

Définir géographiquement les quartiers et rattacher les contenus à ces zones.

7.2 Données quartier

```
id
name
description
geoJsonPolygon
city
postalCode
createdById
isActive
createdAt
updatedAt
archivedAt
```

7.3 Fonctionnalités

P0

- Créer un quartier.
- Lister les quartiers.
- Consulter un quartier.
- Modifier un quartier.
- Archiver un quartier.

P1

- Dessiner la zone sur une carte.
- Vérifier si un point appartient au quartier.
- Lister les habitants d'un quartier.
- Filtrer services/événements/votes par quartier.

P2

- Fusionner deux quartiers.
 - Détecter chevauchements.
 - Gérer les frontières avec tolérance.
-

7.4 Règles métier

- Un quartier archivé ne doit plus recevoir de nouveaux services.
 - Un utilisateur appartient à un quartier principal.
 - Les services, votes et événements sont visibles en priorité dans le quartier de l'utilisateur.
 - Un admin peut consulter tous les quartiers.
 - Un habitant ne doit pas pouvoir modifier un quartier.
-

7.5 Endpoints recommandés

```
POST   /api/neighborhoods
GET    /api/neighborhoods
GET    /api/neighborhoods/:id
PATCH /api/neighborhoods/:id
DELETE /api/neighborhoods/:id
POST   /api/neighborhoods/:id/contains-point
GET    /api/neighborhoods/:id/members
GET    /api/neighborhoods/:id/stats
```

8. Module Services avancé

8.1 Objectif

Le module services est le cœur du projet.

Il doit permettre aux habitants de publier des offres ou demandes, de recevoir des candidatures, de négocier, de signer un contrat, de réaliser le service, de valider le résultat, de transférer les points et de gérer les litiges.

8.2 Types de services

Offre

Un habitant propose une compétence.

Exemples :

```
Je peux garder des enfants.
Je peux aider à monter un meuble.
Je peux donner des cours de maths.
Je peux arroser vos plantes.
```

Demande

Un habitant cherche de l'aide.

Exemples :

Je cherche quelqu'un pour garder mon chat.
J'ai besoin d'aide pour déménager.
Je cherche quelqu'un pour m'aider à réparer une fuite.

8.3 Nature économique du service

Un service peut être :

free
paid
mixed
deposit_required
recurring
urgent

Gratuit

Aucun contrat obligatoire, mais le service reste historisé.

Payant

Contrat obligatoire et points réservés.

Mixte

Cas avancé :

Le service est gratuit mais une caution est demandée.
Le service est payé partiellement en points.
Le service peut contenir des frais variables.

Récurrent

Exemple :

Aide aux courses chaque samedi.
Cours hebdomadaire.
Arrosage des plantes pendant deux semaines.

Urgent

Un service urgent doit être mis en avant et peut déclencher des notifications.

8.4 Données d'un service

```
id
type
title
description
category
neighborhoodId
ownerId
locationType
locationLabel
availability
isPaid
pricePoints
estimatedDuration
isUrgent
isRecurring
recurrenceRule
status
selectedApplicationId
contractId
conversationId
createdAt
updatedAt
publishedAt
completedAt
cancelledAt
```

8.5 États du service

```
draft
published
open_to_applications
application_received
negotiation
candidate_selected
contract_pending
```



```
awaiting_signatures
contract_active
scheduled
in_progress
awaiting_validation
completed
cancelled
disputed
archived
```

8.6 Workflow complet

Étape 1 — Brouillon

L'utilisateur crée un service en brouillon.

Règles :

- visible uniquement par le créateur ;
- modifiable librement ;
- pas de candidatures ;
- pas de contrat ;
- pas de points réservés.

Étape 2 — Publication

L'utilisateur publie le service.

Règles :

- le service devient visible dans le quartier ;
- il apparaît dans la recherche ;
- les voisins peuvent candidater ;
- une notification peut être envoyée aux voisins intéressés par la catégorie.

Étape 3 — Candidatures

Un ou plusieurs voisins candidatent.

Une candidature peut contenir :

```
message
date proposée
prix proposé
```

durée estimée
conditions

Règles :

- impossible de candidater à son propre service ;
- une candidature peut être retirée ;
- le créateur du service peut accepter, refuser ou négocier ;
- un service peut recevoir plusieurs candidatures.

Étape 4 — Négociation

Les parties peuvent négocier.

Exemples :

Je peux venir vendredi au lieu de samedi.
Je peux le faire pour 40 points au lieu de 50.
Je peux faire seulement une partie du service.

Règles :

- chaque contre-proposition est versionnée ;
- une seule proposition active à la fois ;
- l'acceptation d'une proposition verrouille les conditions ;
- l'historique est conservé.

Étape 5 — Sélection du candidat

Le demandeur choisit une candidature.

Conséquences :

- les autres candidatures passent en refusées ;
- une conversation dédiée est créée si elle n'existe pas ;
- si le service est payant, un contrat est préparé ;
- les points sont vérifiés.

Étape 6 — Génération du contrat

Pour un service payant, le système crée un contrat.

Le contrat reprend :

demandeur
prestataire
description du service
prix
date
conditions
politique d'annulation

Règles :

- un service payant ne peut pas avancer sans contrat ;
- le contrat doit être accepté par les deux parties ;
- les points sont réservés avant activation.

Étape 7 — Signature

Les deux parties signent.

Règles :

- signature MFA si disponible ;
- audit obligatoire ;
- le contrat devient actif seulement quand les deux parties ont signé ;
- après signature, les conditions sont verrouillées.

Étape 8 — Planification

Le service est planifié.

Règles :

- date et heure confirmées ;
- rappels envoyés ;
- replanification possible uniquement avec accord des deux parties ;
- chaque changement est historisé.

Étape 9 — Réalisation

Le prestataire démarre et réalise le service.

Possibilités :

- marquer comme démarré ;
- ajouter des notes ;
- envoyer des preuves ;
- demander validation.

Étape 10 — Validation

Le demandeur peut :

valider
demander correction
contester
ne pas répondre

Règles :

- validation déclenche transfert des points ;
- absence de réponse peut déclencher validation automatique après délai ;
- contestation ouvre un litige.

Étape 11 — Clôture

Quand le service est validé :

- les points sont transférés ;
- le contrat passe en terminé ;
- le service passe en terminé ;
- les parties peuvent laisser un avis ;
- Neo4j est mis à jour.

8.7 Endpoints services

```
POST    /api/services
GET      /api/services
GET      /api/services/:id
PATCH   /api/services/:id
DELETE   /api/services/:id

POST     /api/services/:id/publish
POST     /api/services/:id/cancel
POST     /api/services/:id/archive

GET      /api/services/me/created
GET      /api/services/me/involved
GET      /api/services/categories

POST     /api/services/:id/start
POST     /api/services/:id/mark-done
```

```
POST /api/services/:id/validate
POST /api/services/:id/dispute
```

9. Module Candidatures

9.1 Objectif

Permettre à plusieurs voisins de proposer leur aide avant qu'un prestataire soit choisi.

9.2 Données candidature

```
id
serviceId
candidateId
message
proposedDate
proposedPricePoints
estimatedDuration
status
createdAt
updatedAt
withdrawnAt
```

9.3 États candidature

```
submitted
viewed
shortlisted
negotiating
accepted
rejected
withdrawn
expired
```

9.4 Règles métier

- Un utilisateur ne peut pas candidater à son propre service.
- Un utilisateur ne peut avoir qu'une candidature active par service.
- Le créateur du service peut refuser une candidature.
- Le candidat peut retirer sa candidature tant qu'elle n'est pas acceptée.

- Accepter une candidature refuse automatiquement les autres.
 - Une candidature acceptée sert de base au contrat.
 - Une candidature expirée ne peut plus être acceptée.
-

9.5 Endpoints candidatures

```
POST    /api/services/:serviceId/applications
GET     /api/services/:serviceId/applications
GET     /api/applications/me
GET     /api/applications/:id
PATCH  /api/applications/:id
POST    /api/applications/:id/accept
POST    /api/applications/:id/reject
POST    /api/applications/:id/withdraw
POST    /api/applications/:id/shortlist
```

10. Module Propositions et négociation

10.1 Objectif

Permettre aux deux parties d'ajuster les conditions avant contrat.

10.2 Données proposition

```
id
applicationId
serviceId
authorId
pricePoints
scheduledAt
durationMinutes
locationLabel
message
version
status
createdAt
expiresAt
```

10.3 États proposition

```
pending
accepted
rejected
expired
cancelled
superseded
```

10.4 Règles métier

- Une proposition appartient à une candidature.
- Une nouvelle contre-proposition remplace la précédente.
- Les anciennes propositions restent consultables.
- Une proposition acceptée verrouille les conditions du futur contrat.
- Une proposition expirée ne peut plus être acceptée.

10.5 Endpoints propositions

```
POST  /api/applications/:applicationId/proposals
GET    /api/applications/:applicationId/proposals
POST   /api/proposals/:id/accept
POST   /api/proposals/:id/reject
POST   /api/proposals/:id/cancel
```

11. Module Contrats

11.1 Objectif

Formaliser les engagements entre deux voisins.

11.2 Données contrat

```
id
serviceId
applicationId
requesterId
providerId
payerId
receiverId
```

```
pricePoints
description
scheduledAt
durationMinutes
terms
cancellationPolicy
status
signedByIds
signedAt
completedAt
cancelledAt
disputeId
documentId
createdAt
updatedAt
```

11.3 États contrat

```
draft
generated
sent
viewed_by_requester
viewed_by_provider
signed_by_requester
signed_by_provider
active
completed
cancelled
disputed
expired
archived
```

11.4 Règles métier

- Un service payant doit avoir un contrat.
- Le contrat est généré depuis une candidature acceptée.
- Les points doivent être réservés.
- Les deux parties doivent signer.
- Le contrat devient actif quand les deux signatures sont présentes.
- Les conditions ne peuvent plus être changées après activation.
- Toute modification après activation nécessite un avenant.
- Un contrat expiré libère les points réservés.
- Un contrat en litige gèle les points.

11.5 Avenants

Un avenant permet de modifier un contrat actif sans détruire l'historique.

Cas d'usage :

```
changer la date
changer le prix
changer la durée
changer les conditions
```

États d'un avenant :

```
draft
sent
accepted
rejected
expired
```

11.6 Endpoints contrats

```
POST  /api/contracts/from-application/:applicationId
GET    /api/contracts
GET    /api/contracts/:id
POST   /api/contracts/:id/send
POST   /api/contracts/:id/sign
POST   /api/contracts/:id/complete
POST   /api/contracts/:id/cancel
POST   /api/contracts/:id/dispute
GET    /api/contracts/:id/audit

POST   /api/contracts/:id/amendments
GET    /api/contracts/:id/amendments
POST   /api/contracts/:id/amendments/:amendmentId/accept
POST   /api/contracts/:id/amendments/:amendmentId/reject
```

12. Module Points

12.1 Objectif

Gérer une monnaie interne fiable.

12.2 Soldes utilisateur

Chaque utilisateur possède :

```
availablePoints  
reservedPoints  
frozenPoints  
earnedPoints  
spentPoints
```

12.3 Types de transactions

```
initial_credit  
reservation  
transfer  
release  
refund  
penalty  
bonus  
admin_adjustment  
dispute_freeze  
dispute_release
```

12.4 Règles métier

- Les points disponibles peuvent être réservés.
 - Les points réservés ne peuvent pas être réutilisés.
 - Les points gelés ne peuvent pas être transférés.
 - Chaque opération crée une transaction.
 - Une transaction ne doit jamais être supprimée.
 - Toute correction se fait par transaction inverse.
 - Les opérations de points doivent être atomiques.
-

12.5 Scénario normal

1. Alice accepte un contrat payant.
2. 50 points sont réservés.
3. Le service est réalisé.
4. Alice valide.
5. Les 50 points réservés sont transférés à Bob.
6. Une transaction de transfert est créée.

12.6 Scénario annulation

1. Alice annule avant signature.
2. Les points réservés sont libérés.
3. Le contrat passe en annulé.
4. Une transaction release est créée.

12.7 Scénario litige

1. Bob déclare le service terminé.
2. Alice conteste.
3. Les points réservés deviennent gelés.
4. Un litige est ouvert.
5. Le modérateur décide.
6. Les points sont transférés, remboursés ou partagés.

12.8 Endpoints points

```
GET    /api/points/balance
GET    /api/points/transactions
GET    /api/points/transactions/:id

POST   /api/points/reserve
POST   /api/points/release
POST   /api/points/transfer-reserved
POST   /api/points/freeze
POST   /api/points/unfreeze

POST   /api/admin/points/adjust
GET    /api/admin/points/users/:userId
```

13. Module Exécution du service

13.1 Objectif

Suivre ce qui se passe entre la signature et la validation.

13.2 États d'exécution

```
scheduled  
started  
step_completed  
proof_uploaded  
provider_marked_done  
requester_validated  
auto_validated  
correction_requested  
disputed  
completed
```

13.3 Preuves

Le prestataire peut ajouter :

```
photo  
document  
message  
note  
audio  
timestamp
```

Exemples :

```
Photo du meuble monté.  
Photo des plantes arrosées.  
Document rempli.  
Message de fin d'intervention.
```

13.4 Validation

Le demandeur peut :

```
valider  
demander une correction  
contester  
ignorer
```

Si le demandeur ne répond pas :

rappel automatique
puis validation automatique après délai
sauf litige ouvert

13.5 Endpoints

POST /api/services/:id/start
POST /api/services/:id/proofs
GET /api/services/:id/proofs
POST /api/services/:id/mark-done
POST /api/services/:id/validate
POST /api/services/:id/request-correction

14. Module Annulations

14.1 Objectif

Gérer proprement les annulations selon le moment où elles arrivent.

14.2 Cas d'annulation

Avant candidature

service annulé
aucune conséquence financière

Après candidature acceptée

candidature annulée
proposition annulée
notification envoyée

Après contrat généré

contrat annulé ou expiré
points libérés
audit créé

Après signature

application de la politique d'annulation
possible pénalité
historique conservé

Pendant exécution

ouverture recommandée d'un litige
points gelés
modérateur notifié

14.3 Politique d'annulation

Une politique peut définir :

freeCancellationUntil
penaltyPoints
requiresModerator
autoRefund

14.4 Endpoints

POST /api/services/:id/cancel
POST /api/contracts/:id/cancel
GET /api/cancellation-policies
POST /api/admin/cancellation-policies

15. Module Litiges

15.1 Objectif

Résoudre les conflits entre voisins.

15.2 Cas typiques

service non réalisé
service mal réalisé

```
prestataire absent  
demandeur refuse de valider  
annulation abusive  
comportement problématique  
preuves contradictoires
```

15.3 Données litige

```
id  
serviceId  
contractId  
openedById  
againstUserId  
reason  
description  
status  
assignedModeratorId  
decision  
createdAt  
resolvedAt  
closedAt
```

15.4 États litige

```
opened  
evidence_collection  
moderator_review  
decision_pending  
resolved_refund  
resolved_transfer  
resolved_partial  
rejected  
closed
```

15.5 Preuves de litige

Une preuve peut être :

```
message  
image  
document  
audio
```

```
system_event
contract
signature
proof_uploaded
```

15.6 Règles métier

- Ouvrir un litige gèle les points.
- Les deux parties peuvent ajouter des preuves.
- Le modérateur peut consulter les messages liés.
- Le modérateur peut décider :
 - remboursement total ;
 - transfert total ;
 - partage des points ;
 - pénalité ;
 - clôture sans suite.
- La décision crée automatiquement les transactions nécessaires.
- Le litige reste dans l'historique.

15.7 Endpoints litiges

```
POST    /api/disputes
GET     /api/disputes
GET     /api/disputes/:id
POST    /api/disputes/:id/evidence
POST    /api/disputes/:id/assign
POST    /api/disputes/:id/resolve
POST    /api/disputes/:id/close
GET     /api/disputes/:id/audit
```

16. Module Avis et réputation

16.1 Objectif

Créer de la confiance entre voisins.

16.2 Avis

Après un service terminé, chaque partie peut laisser :


```
rating
comment
punctuality
quality
communication
respect
```

16.3 Règles métier

- Avis possible uniquement après service terminé.
- Un utilisateur ne peut laisser qu'un avis par contrat.
- Les avis peuvent être signalés.
- Un avis signalé peut être masqué.
- Les avis influencent le score de réputation.
- Les avis peuvent alimenter Neo4j.

16.4 Score de réputation

Le score peut prendre en compte :

```
nombre de services terminés
note moyenne
litiges perdus
annulations tardives
ancienneté
participation aux événements
signalements reçus
```

16.5 Endpoints

```
POST  /api/reviews
GET    /api/users/:id/reviews
GET    /api/users/:id/reputation
POST   /api/reviews/:id/report
POST   /api/admin/reviews/:id/hide
```

17. Module Documents et signatures

17.1 Objectif

Gérer les PDF, les zones de signature, les signatures et l'archivage.

17.2 Types de documents

```
contract  
amendment  
incident_report  
event_document  
authorization  
other
```

17.3 Cycle de vie document

```
draft  
uploaded  
prepared  
sent_for_signature  
partially_signed  
signed  
finalized  
archived  
cancelled
```

17.4 Types de champs

```
signature  
initials  
date  
text  
checkbox
```

17.5 Données document

```
id
ownerId
contextType
contextId
originalObjectKey
signedObjectKey
mimeType
status
fields
auditTrail
createdAt
updatedAt
finalizedAt
```

17.6 Règles métier

- Le PDF original est conservé.
- Le PDF signé est généré séparément.
- Un champ signé ne peut plus être modifié.
- Un document signé ne peut plus être préparé.
- Chaque signature crée une trace d'audit.
- La signature doit être protégée par MFA si disponible.
- Le document final doit être téléchargeable.

17.7 Endpoints documents

```
POST   /api/documents
GET     /api/documents
GET     /api/documents/:id
DELETE  /api/documents/:id

POST    /api/documents/:id/fields
PATCH  /api/documents/:id/fields/:fieldId
DELETE  /api/documents/:id/fields/:fieldId

POST    /api/documents/:id/send
POST    /api/documents/:id/fields/:fieldId/sign
POST    /api/documents/:id/finalize

GET     /api/documents/:id/download/original
GET     /api/documents/:id/download/signed
GET     /api/documents/:id/audit
```

18. Module Stockage MinIO

18.1 Objectif

Stocker les fichiers lourds hors MongoDB.

18.2 Fichiers concernés

```
PDF originaux
PDF signés
images de messagerie
messages vocaux
preuves de litige
pièces jointes
avatars
documents incidents
```

18.3 Fonctionnement

1. Le client demande une URL d'upload.
 2. L'API vérifie les droits.
 3. L'API génère une URL temporaire MinIO.
 4. Le client upload directement vers MinIO.
 5. L'API enregistre les métadonnées en MongoDB.
-

18.4 Endpoints

```
POST    /api/storage/presign-upload
POST    /api/storage/presign-download
GET     /api/storage/files/:id
DELETE  /api/storage/files/:id
```

19. Module Messagerie

19.1 Objectif

Permettre aux voisins de communiquer dans un contexte sécurisé.

19.2 Contextes de conversation

Une conversation peut être liée à :

```
service
application
contract
event
dispute
incident
direct
```

19.3 Types de messages

```
text
image
audio
document
system
contract_event
payment_event
moderation_event
```

19.4 Messages système

Exemples :

```
Alice a créé une demande.
Bob a candidaté.
Alice a accepté la candidature de Bob.
Le contrat a été envoyé.
Alice a signé le contrat.
Les points ont été réservés.
Bob a marqué le service comme réalisé.
Alice a ouvert un litige.
```

19.5 Règles métier

- Seuls les participants peuvent lire une conversation.
- Un modérateur peut accéder à une conversation uniquement dans un litige ou signalement.
- Les messages système ne peuvent pas être modifiés.

- Les fichiers passent par MinIO.
- Les messages liés à un litige peuvent être gelés comme preuves.
- Les messages supprimés peuvent être anonymisés mais pas forcément détruits si preuve.

19.6 Endpoints REST

```
POST    /api/conversations
GET     /api/conversations
GET     /api/conversations/:id

POST    /api/conversations/:id/messages
GET     /api/conversations/:id/messages
POST    /api/conversations/:id/attachments
POST    /api/conversations/:id/read

POST    /api/messages/:id/report
DELETE  /api/messages/:id
```

19.7 WebSocket

Événements :

```
conversation.join
conversation.leave
message.send
message.new
message.read
typing.start
typing.stop
presence.online
presence.offline
```

20. Module Notifications

20.1 Objectif

Informer les utilisateurs au bon moment.

20.2 Types de notifications

```
service_created
application_received
application_accepted
application_rejected
proposal_received
contract_sent
contract_signed
contract_active
service_scheduled
service_started
service_done
validation_required
points_reserved
points_transferred
dispute_opened
dispute_resolved
event_created
event_reminder
vote_created
vote_closing_soon
message_received
incident_updated
sync_completed
```

20.3 Canaux

```
in_app
email
websocket
desktop_javafx
```

20.4 Endpoints

```
GET    /api/notifications
GET    /api/notifications/unread-count
PATCH /api/notifications/:id/read
PATCH /api/notifications/read-all
GET    /api/notification-preferences
PATCH /api/notification-preferences
```

21. Module Événements

21.1 Objectif

Organiser la vie du quartier.

21.2 Types d'événements

workshop
party
fundraising
sport
community_meeting
children
culture
help
emergency
other

21.3 États événement

draft
published
open_registration
full
started
completed
cancelled
archived

21.4 Réponses utilisateur

interested
going
maybe
not_interested
cancelled
waitlisted

21.5 Fonctionnalités

- Créer un événement.
 - Publier un événement.
 - Gérer une capacité maximale.
 - Rejoindre l'événement.
 - Se désinscrire.
 - Liste d'attente.
 - Rappels automatiques.
 - Recommandation.
 - Statistiques de participation.
-

21.6 Endpoints événements

```
POST    /api/events
GET     /api/events
GET     /api/events/:id
PATCH  /api/events/:id
DELETE  /api/events/:id

POST    /api/events/:id/publish
POST    /api/events/:id/respond
POST    /api/events/:id/join
POST    /api/events/:id/leave
GET     /api/events/:id/participants
GET     /api/events/me/responses
GET     /api/events/recommended
```

22. Module Recommandations Neo4j

22.1 Objectif

Utiliser les relations sociales pour proposer des contenus pertinents.

22.2 Recommandations possibles

```
événements recommandés
services recommandés
voisins fiables
candidats recommandés
catégories populaires
```

22.3 Critères

Pour recommander un voisin :

```
services terminés  
bonnes notes  
peu de litiges  
proximité quartier  
catégories communes  
événements communs
```

Pour recommander un événement :

```
même quartier  
catégorie déjà aimée  
amis/voisins participants  
popularité  
historique utilisateur
```

22.4 Endpoints

```
GET    /api/recommendations/events  
GET    /api/recommendations/services  
GET    /api/recommendations/neighbors  
GET    /api/recommendations/services/:id/candidates
```

23. Module Votes

23.1 Objectif

Permettre aux habitants de voter sur la vie du quartier.

23.2 Types de votes

```
yes_no  
single_choice  
multiple_choice  
ranking
```

```
anonymous
public
```

23.3 États vote

```
draft
scheduled
open
closed
cancelled
archived
```

23.4 Règles métier

- Un vote appartient à un quartier.
 - Seuls les habitants concernés peuvent voter.
 - Un utilisateur ne peut voter qu'une fois sauf modification autorisée.
 - Certains votes sont anonymes.
 - Certains résultats sont visibles uniquement après clôture.
 - Un vote clôturé ne peut plus recevoir de réponse.
-

23.5 Endpoints votes

```
POST    /api/votes
GET      /api/votes
GET      /api/votes/:id
PATCH   /api/votes/:id
POST     /api/votes/:id/open
POST     /api/votes/:id/answers
GET      /api/votes/:id/results
POST     /api/votes/:id/close
POST     /api/votes/:id/cancel
```

24. Module Incidents et alertes

24.1 Objectif

Permettre la gestion des problèmes de quartier, notamment via JavaFX.

24.2 Types d'incidents

```
security
maintenance
noise
cleanliness
technical
neighbor_conflict
event_issue
other
```

24.3 États incident

```
reported
open
assigned
in_progress
waiting_information
resolved
closed
rejected
archived
```

24.4 Alertes

Une alerte est liée à un incident.

Sévérités :

```
low
medium
high
critical
```

24.5 Règles métier

- Un incident peut contenir plusieurs alertes.
- Une alerte critique doit notifier un admin.
- Un incident créé offline doit être synchronisé.
- Un incident modifié localement et serveur peut créer un conflit.
- Les statistiques doivent pouvoir être consultées offline.

24.6 Endpoints incidents

```
POST    /api/incidents
GET     /api/incidents
GET     /api/incidents/:id
PATCH  /api/incidents/:id
DELETE  /api/incidents/:id

POST    /api/incidents/:id/start
POST    /api/incidents/:id/resolve
POST    /api/incidents/:id/close

POST    /api/incidents/:id/alerts
GET     /api/incidents/:id/alerts
PATCH  /api/alerts/:id
DELETE  /api/alerts/:id
```

25. Application JavaFX offline-first

25.1 Objectif

Permettre à l'administrateur de travailler même sans connexion Internet.

25.2 Fonctionnalités JavaFX

P0

- Lancer l'application.
- Consulter les incidents locaux.
- Créer un incident.
- Modifier un statut.
- Ajouter une alerte.
- Consulter statistiques simples.
- Stocker en base locale.

P1

- Outbox de synchronisation.
- Pull depuis l'API.
- Push vers l'API.
- Détection conflits.
- JAR exécutable.

P2

- Plugins.
 - Thèmes.
 - Mises à jour automatiques.
 - Désinstallation depuis l'interface.
-

25.3 Tables locales

```
incidents  
alerts  
reporters  
sync_outbox  
sync_state  
sync_conflicts  
themes  
plugins
```

26. Synchronisation JavaFX/API

26.1 Objectif

Synchroniser les données locales JavaFX avec l'API centrale.

26.2 Outbox

Chaque action offline crée une opération :

```
operationId  
entityType  
entityId  
operationType  
payload  
createdAt  
status  
retryCount  
lastError
```

26.3 Processus de synchronisation

1. Vérifier connexion API.
2. Envoyer les opérations locales.
3. Recevoir les confirmations serveur.
4. Récupérer les changements serveur depuis lastSyncAt.
5. Appliquer les changements localement.
6. Détecter les conflits.
7. Mettre à jour lastSyncAt.

26.4 Endpoints sync

```
POST    /api/sync/push
GET     /api/sync/pull?since=...
POST    /api/sync/conflicts/:id/resolve
GET     /api/sync/status
GET     /api/sync/history
```

26.5 Stratégie de conflit

V1 :

```
server_wins
client_wins
manual_resolution
```

Règle par défaut :

```
Dernière modification gagnante
+ journal d'audit
+ conflit visible en admin si doute
```

27. Module Modération

27.1 Objectif

Gérer les contenus problématiques.

27.2 Contenus modérables

```
services
messages
reviews
events
votes
profiles
documents metadata
```

27.3 Signalements

Raisons possibles :

```
spam
scam
abusive_behavior
harassment
inappropriate_content
fake_service
dangerous_content
privacy_issue
```

27.4 États signalement

```
submitted
under_review
action_taken
rejected
closed
```

27.5 Actions modération

```
hide_content
warn_user
suspend_user
close_service
cancel_event
delete_message
```



```
freeze_contract  
open_dispute
```

27.6 Endpoints

```
POST    /api/reports  
GET     /api/admin/reports  
GET     /api/admin/reports/:id  
POST    /api/admin/reports/:id/assign  
POST    /api/admin/reports/:id/resolve  
POST    /api/admin/moderation/actions
```

28. Module RGPD

28.1 Objectif

Respecter les droits des utilisateurs.

28.2 Données concernées

```
profil  
services  
candidatures  
propositions  
contrats  
documents  
signatures  
transactions  
messages  
événements  
votes  
avis  
signalements  
incidents  
notifications  
audit logs
```

28.3 Fonctionnalités

P0

- Export des données.
- Anonymisation compte.
- Suppression logique.

P1

- Rectification.
- Historique des demandes.
- Statut d'une demande RGPD.

P2

- Export ZIP.
- Suppression automatisée différée.
- Tableau admin RGPD.

28.4 Endpoints RGPD

```
GET    /api/rgpd/export
POST   /api/rgpd/request-deletion
POST   /api/rgpd/anonymize
PATCH /api/rgpd/profile
GET    /api/rgpd/audit
GET    /api/admin/rgpd/requests
POST   /api/admin/rgpd/requests/:id/process
```

29. Micro-langage MongoDB

29.1 Objectif

Créer un langage d'interrogation maison pour manipuler MongoDB.

29.2 Exemple de syntaxe

```
FIND services WHERE category = "bricolage" AND pricePoints <= 50
FIND events WHERE category = "sport" AND neighborhoodId = "quartier-centre"
FIND votes WHERE status = "open"
```

29.3 Fonctionnement

1. Réception de la requête DSL.
2. Analyse lexicale.
3. Analyse syntaxique.
4. Génération d'un AST.
5. Validation sécurité.
6. Traduction en filtre MongoDB.
7. Exécution.
8. Retour résultat.

29.4 Collections autorisées

```
services
events
votes
documents
incidents
```

29.5 Endpoints DSL

```
POST  /api/dsl/parse
POST  /api/dsl/execute
GET    /api/dsl/examples
```

30. Front web utilisateur

30.1 Pages

```
/login
/dashboard
/services
/services/:id
/services/:id/applications
/applications/me
/contracts
/contracts/:id
/documents
/events
/votes
```

```
/messages  
/profile  
/notifications  
/rgpd
```

30.2 Parcours principal

1. Connexion.
2. Création d'une demande de service payante.
3. Réception de candidatures.
4. Consultation des profils candidats.
5. Négociation.
6. Acceptation.
7. Génération du contrat.
8. Signature.
9. Réalisation.
10. Validation.
11. Transfert points.
12. Avis.

31. Back-office React

31.1 Pages

```
/admin/dashboard  
/admin/users  
/admin/neighborhoods  
/admin/services  
/admin/contracts  
/admin/disputes  
/admin/reports  
/admin/events  
/admin/votes  
/admin/documents  
/admin/incidents  
/admin/statistics  
/admin/sync  
/admin/settings
```

31.2 Dashboard

Afficher :

```
utilisateurs actifs  
services publiés  
contrats en cours  
litiges ouverts  
incidents critiques  
événements à venir  
votes ouverts  
documents en attente  
transactions de points  
état synchronisation JavaFX
```

32. Docker et déploiement

32.1 Services Docker

```
api  
web  
admin-web  
mongodb  
neo4j  
minio  
keycloak
```

32.2 Fichiers nécessaires

```
docker-compose.yml  
apps/api/Dockerfile  
apps/web/Dockerfile  
apps/admin-web/Dockerfile  
.env.example  
README_DEPLOIEMENT.md
```

33. Tests

33.1 Tests API

```
services  
applications  
proposals  
contracts
```

```
points
documents
events
votes
messaging
incidents
sync
rgpd
dsl
```

33.2 Tests front

```
login
création service
candidature
signature contrat
validation service
votes
événements
messages
```

33.3 Tests JavaFX

```
création incident
modification incident
ajout alerte
stats
base locale
outbox
sync
conflits
packaging JAR
```

34. Documentation

34.1 Dossier technique

Inclure :

```
architecture globale
schémas
```

- modélisation MongoDB
- modélisation Neo4j
- workflows métier
- sécurité
- RGPD
- synchronisation
- micro-langage
- tests
- déploiement
- choix techniques
- limites

34.2 Dossier utilisateur

Inclure :

- installation
- lancement
- connexion
- utilisation web
- utilisation admin web
- utilisation JavaFX
- scénarios de démonstration
- comptes de test
- dépannage

34.3 Synthèse critique

Inclure :

- démarche
- répartition du travail
- difficultés
- choix techniques
- fonctionnalités terminées
- fonctionnalités partielles
- limites
- améliorations futures

35. Roadmap de réalisation

Phase 1 — Stabilisation

- ☐ Vérifier build API.
 - ☐ Vérifier tests API.
 - ☐ Vérifier front web.
 - ☐ Vérifier branche JavaFX.
 - ☐ Corriger tests obsolètes.
 - ☐ Nettoyer logs debug.
 - ☐ Mettre à jour README.
-

Phase 2 — Services avancés

- ☐ Ajouter candidatures.
 - ☐ Ajouter propositions.
 - ☐ Ajouter négociation.
 - ☐ Ajouter sélection candidat.
 - ☐ Ajouter états avancés.
 - ☐ Ajouter historique service.
 - ☐ Ajouter notifications.
 - ☐ Ajouter annulations.
 - ☐ Ajouter avis.
-

Phase 3 — Contrats et points avancés

- ☐ Contrat depuis candidature.
 - ☐ Signature double.
 - ☐ Expiration contrat.
 - ☐ Avenants.
 - ☐ Gel points litige.
 - ☐ Transactions avancées.
 - ☐ Audit complet.
-

Phase 4 — Documents et MinIO

- ☐ Ajouter MinIO.
 - ☐ Presigned upload.
 - ☐ Presigned download.
 - ☐ PDF original.
 - ☐ PDF signé.
 - ☐ Audit signatures.
-

Phase 5 — Litiges et modération

- ☐ Créer module litiges.
 - ☐ Ajouter preuves.
 - ☐ Ajouter décisions modérateur.
 - ☐ Ajouter signalements.
 - ☐ Ajouter actions modération.
-

Phase 6 — JavaFX et synchronisation

- ☐ Fusionner branche JavaFX.
 - ☐ Corriger lancement.
 - ☐ Ajouter endpoints incidents.
 - ☐ Ajouter sync push/pull.
 - ☐ Ajouter outbox.
 - ☐ Ajouter conflits.
 - ☐ Générer JAR.
-

Phase 7 — Admin web

- ☐ Remplacer template Vite.
 - ☐ Dashboard.
 - ☐ Utilisateurs.
 - ☐ Quartiers.
 - ☐ Services.
 - ☐ Litiges.
 - ☐ Incidents.
 - ☐ Stats.
-

Phase 8 — Neo4j

- ☐ Service Neo4j.
 - ☐ Modèle graphe.
 - ☐ Écriture interactions.
 - ☐ Recommandations événements.
 - ☐ Recommandations services.
 - ☐ Recommandations voisins.
-

Phase 9 — Micro-langage

- ☐ Grammaire.
- ☐ Parser.
- ☐ AST.
- ☐ Traduction MongoDB.

- [] Endpoints.
 - [] Tests.
 - [] Documentation.
-

Phase 10 — Finalisation

- [] Docker Compose complet.
 - [] `.env.example`.
 - [] Swagger propre.
 - [] Jeux de données.
 - [] Documentation technique.
 - [] Documentation utilisateur.
 - [] Synthèse critique.
 - [] Démo complète.
 - [] Nettoyage dépôt.
-

36. Scénario de démonstration final

36.1 Parcours service complet

1. Alice se connecte.
2. Alice crée une demande de service payante.
3. Bob candidate.
4. Bob fait une contre-proposition.
5. Alice accepte.
6. Le contrat est généré.
7. Les points sont réservés.
8. Alice signe.
9. Bob signe.
10. Le service est planifié.
11. Bob marque le service comme réalisé.
12. Bob ajoute une preuve.
13. Alice valide.
14. Les points sont transférés.
15. Les deux laissent un avis.
16. Neo4j met à jour la relation de confiance.

36.2 Parcours litige

1. Bob déclare le service terminé.
2. Alice conteste.
3. Les points sont gelés.
4. Un litige est ouvert.

5. Les preuves sont ajoutées.
6. Le modérateur tranche.
7. Les points sont transférés, remboursés ou partagés.
8. Le litige est clôturé.

36.3 Parcours JavaFX offline

1. Admin lance JavaFX.
2. Les incidents locaux s'affichent.
3. Internet est coupé.
4. Admin crée un incident.
5. L'opération est stockée dans l'outbox.
6. Internet revient.
7. L'application synchronise.
8. L'incident apparaît côté API et admin web.

36.4 Parcours micro-langage

1. Admin ouvre l'outil DSL.
2. Il saisit une requête.
3. Le parser génère un AST.
4. L'API traduit en filtre MongoDB.
5. Les résultats sont affichés.

37. Priorités si le temps est limité

Si le temps manque, l'ordre de priorité doit être :

1. Services avec candidatures.
2. Contrats avec signatures.
3. Points avec réservation/transfert.
4. JavaFX incidents offline.
5. Synchronisation JavaFX minimale.
6. Back-office admin minimal.
7. Documents PDF V1.
8. Litiges simples.
9. Votes.
10. Événements.
11. RGPD export/anonymisation.
12. Micro-langage MongoDB.
13. Neo4j recommandations simples.

- 14. MinIO.
- 15. Messagerie temps réel.

Le minimum défendable n'est pas un CRUD.

Le minimum défendable est un workflow complet qui montre :

```
interaction
engagement
signature
paiement interne
preuve
validation
litige
audit
```

38. Conclusion

Connected Neighbours doit être présenté comme une plateforme de confiance entre voisins.

Le module le plus important est le module services, car il relie :

```
utilisateurs
quartiers
candidatures
négociation
contrats
points
documents
signatures
messagerie
notifications
avis
litiges
modération
recommandations
RGPD
```

Le projet sera convaincant si la soutenance montre un parcours complet, cohérent et traçable.

L'objectif final n'est pas seulement de dire :

```
Nous avons fait une API, un front et une application Java.
```

L'objectif est de montrer :

Nous avons conçu une plateforme complète de quartier, avec des workflows réalistes, une logique métier solide, une administration offline-first et une architecture extensible.